

Supplementary Material for the Article: Mechanically Verified Optimization and UPPAAL Export for Timed Büchi Automata

ELIE FARES¹, JEAN-PAUL BODEVEIX², HUSSAIN AL-AQRABI¹ AND AZAR SALAMI¹

¹Higher Colleges of Technology, Sharjah, United Arab Emirates (e-mail: efares@hct.ac.ae, halaqrabi@hct.ac.ae, asalami@hct.ac.ae)

²IRIT, UPS, Université de Toulouse, France (e-mail: bodeveix@irit.fr)

ABSTRACT This supplement gives detailed definitions and proof arguments for the verified TBA optimization and UPPAAL export passes, the full benchmark data and formula list, additional implementation checks, and the system models used in the UPPAAL case study. The Coq preservation results quantify over arbitrary TBAs; measurements are limited to benchmark automata recorded in the project.

S1. THE VERIFIED POST-PROCESSING PASSES

This section details the passes summarized in Table 1 of the article.

A. LOCATION INVARIANTS ON THE TRACE MODEL

A location invariant maps every clock x either to an upper bound $M \in \mathbb{R}$, standing for the constraint $x \leq M$, or to $+\infty$ (no constraint). We interpret automata with invariants on the extended words of Section II, as we do for automata without invariants. Along a run r , the automaton stays in location $r(i)$ between the events $i-1$ and i , and the invariant of $r(i)$ must hold during the whole stay, that is, at the clock values $\text{val}_\rho(i, x) - \tau$ for every delay $\tau \in [0, \Delta_{i-1}]$, where $\Delta_{i-1} = t_i - t_{i-1}$ (and at the values $\text{val}_\rho(0, x)$ for the initial location). This is the condition of Section II, read at the event that ends the stay. Since clocks increase while time elapses, an invariant made of upper bounds holds during the stay if and only if it holds at the event that ends it (lemma `inv_during_iff_at_event`). For the two-sided invariants of Section S1-C, a lower bound $x \geq m$ must likewise hold during the whole stay, hence at its beginning.

B. INVARIANT SYNTHESIS AND BACKWARD PROPAGATION

In the exported timed automaton, upper-bounded guards need invariants so that time does not elapse past all useful outgoing transitions. We compute location invariants from outgoing upper bounds and propagate successor requirements backward, following the standard timed-automata intuition of propagating admissible time domains [1].

a: Upper bound of a guard.

For a conjunctive guard g and a clock x , let $ub_x(g)$ be the smallest bound m of the constraints $x \leq m$, $x < m$, and $x = m$ occurring in g , and $+\infty$ if g contains none. Other constraints are ignored. A valuation satisfying g satisfies $x \leq ub_x(g)$.

b: Invariant synthesis.

For a location ℓ with outgoing transitions $\ell \xrightarrow{g_i, Z_i} \ell_i$, $i = 1, \dots, m$, the synthesized invariant is

$$\text{Inv}(\ell)(x) = \max_{1 \leq i \leq m} ub_x(g_i),$$

with $\text{Inv}(\ell)(x) = +\infty$ when ℓ has no outgoing transition. Thus a clock that some outgoing guard does not bound does not appear in the invariant. The bound is always non-strict, even when it comes from a strict guard $x < m$: the invariant only needs to be implied by the guard of the next transition, and strictness is kept in the guards themselves.

Theorem S1.1 (Invariant synthesis). *For every TBA $\mathcal{A}$, the automaton $\mathcal{A}$ with the synthesized invariants accepts the same timed words as $\mathcal{A}$.*

Proof. Adding invariants can only remove runs. Conversely, let r be an accepting run on ρ . At every event i , the transition taken from $r(i)$ is enabled, so its guard holds at the values $\text{val}_\rho(i, x)$, which are therefore at most $\text{Inv}(r(i))(x)$. By the lemma `inv_during_iff_at_event`, the invariant of $r(i)$ holds during the whole stay. (Coq: `add_invariants_correct`.) $\square$

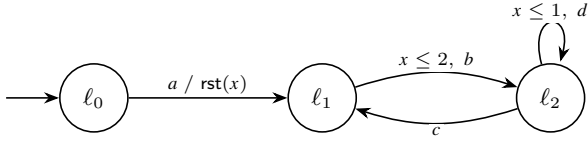


FIGURE S1. Automaton used to illustrate invariant propagation, with one clock x and actions a – d . Edges are labeled “guard, action / resets” (the guard is omitted when it is $\top$); only the edge $\ell_0 \rightarrow \ell_1$ resets x . The incoming arrow marks the initial location; acceptance plays no role here. The computed fixpoint is $Inv(\ell_1) = Inv(\ell_2) = (x \leq 2)$.

c: Backward propagation.

One propagation step strengthens every transition $\ell \xrightarrow{g, Z} \ell'$ with the invariant of its target after the resets:

$$g := g \wedge Inv(\ell')[Z := 0],$$

where $Inv(\ell')[Z := 0]$ replaces every bound $x \leq M$ on a reset clock $x \in Z$ by the constant constraint $0 \leq M$, that is, by $\top$ if $M \geq 0$ and by $\perp$ otherwise. Invariants are then synthesized again from the strengthened guards.

Theorem S1.2 (Backward propagation). *For every TBA $\mathcal{A}$ and every $n \in \mathbb{N}$, the automaton obtained by n propagation steps accepts the same timed words as $\mathcal{A}$.*

Proof. Strengthening guards can only remove runs. Conversely, consider an accepting run that takes $\ell \xrightarrow{g, Z} \ell'$ at event i and then a transition $\ell' \xrightarrow{g', Z'} \ell''$ at event $i + 1$. The second guard holds at event $i + 1$, so the values at that event are bounded by $Inv(\ell')$. Clock consistency gives $\text{val}_\rho(i + 1, x) = \Delta_i$ for $x \in Z$ and $\text{val}_\rho(i + 1, x) = \text{val}_\rho(i, x) + \Delta_i \geq \text{val}_\rho(i, x)$ otherwise; hence the strengthened guard $g \wedge Inv(\ell')[Z := 0]$ holds at event i . The argument uses clock consistency, so the theorem is stated on timed words. (Coq: `propagate_accepts` and `propagate_n_accepts`.) $\square$

Each round of the optimization pipeline of Section III-E applies one propagation step. The tool repeats the rounds until the automaton is structurally equal to the previous one, as an OCaml value, or until 50 rounds have been performed; the bound guarantees termination, and the theorem holds for every number of steps, whether or not a fixpoint is reached. On our benchmarks, the rounds reach a fixpoint after at most five rounds. The invariant computation at one location is summarized in the article’s Figure 1.

d: Example.

We illustrate the algorithm on the automaton of Fig. S1, which contains a two-location cycle $\ell_1 \rightarrow \ell_2 \rightarrow \ell_1$ and a self-loop at ℓ_2 . It has one clock x , the initial location ℓ_0 , and the transitions

$$\begin{aligned} \ell_0 &\xrightarrow{a, Z=\{x\}} \ell_1, & \ell_1 &\xrightarrow{b, x \leq 2, Z=\emptyset} \ell_2, \\ \ell_2 &\xrightarrow{c, \top, Z=\emptyset} \ell_1, & \ell_2 &\xrightarrow{d, x \leq 1, Z=\emptyset} \ell_2. \end{aligned}$$

The edge $\ell_1 \xrightarrow{b} \ell_2$ requires $x \leq 2$, and the self-loop at ℓ_2 requires the tighter bound $x \leq 1$.

- 1) *Synthesis.* $Inv(\ell_1)(x) = ub_x(x \leq 2) = 2$. At ℓ_2 , $Inv(\ell_2)(x) = \max(1, +\infty) = +\infty$, because the transition toward ℓ_1 does not bound x . At ℓ_0 , the only outgoing guard is $\top$, so $Inv(\ell_0)(x) = +\infty$.
- 2) *Propagation.* The transition $\ell_2 \xrightarrow{c, Z=\emptyset} \ell_1$ becomes guarded by $\top \wedge Inv(\ell_1) = (x \leq 2)$. The transition $\ell_0 \xrightarrow{a, Z=\{x\}} \ell_1$ receives $(0 \leq 2) \equiv \top$ and is unchanged.
- 3) *Synthesis.* At ℓ_2 , the outgoing guards are now $x \leq 1$ and $x \leq 2$, so $Inv(\ell_2)(x) = \max(1, 2) = 2$.
- 4) *Propagation.* The transition $\ell_1 \xrightarrow{b, x \leq 2} \ell_2$ receives $x \leq 2$, which it already contains, and the self-loop at ℓ_2 receives $x \leq 2$, which is implied by $x \leq 1$. Nothing changes.

The fixpoint is $Inv(\ell_0) = \top$ and $Inv(\ell_1) = Inv(\ell_2) = (x \leq 2)$. While the automaton remains in the cycle $\ell_1 \leftrightarrow \ell_2$, the clock x cannot exceed 2, and at every reachable valuation at least one outgoing transition of the current location is enabled. The invariants do not remove every blocking behavior: since x is never reset in the cycle, time is blocked once $x = 2$. They make the upper bounds of the property automaton explicit, and Theorems S1.1 and S1.2 guarantee that they do not change its language.

C. FORWARD PROPAGATION OF LOWER BOUNDS AND SIMPLIFICATION

a: Entry bounds.

Dually, for a transition $t = \ell \xrightarrow{g, Z} \ell'$ and a clock x , the *entry bound* of x along t is 0 if $x \in Z$ and otherwise the largest bound $lb_x(g)$ of the constraints $x \geq m$, $x > m$, and $x = m$ of g , if any. The entry bound of x at a location ℓ' is the minimum of the entry bounds of x along the transitions entering ℓ' ; for the initial location, the initial entry, with bound 0, also counts. If some entering transition gives no bound on x , the clock x has no entry bound at ℓ' . Since clocks are nonnegative and do not decrease while time elapses, every clock with entry bound m at ℓ' satisfies $x \geq m$ during every stay in ℓ' .

b: Forward propagation.

The forward step adds $x \geq m$ to the guard of every transition leaving ℓ' , for every clock x with entry bound m at ℓ' (Coq: `forward_accepts`). The entry bounds can also be read as lower-bound invariants: the two-sided automaton that carries the synthesized upper-bound invariants and the entry bounds as lower-bound invariants accepts the same timed words (`add_two_sided_invariants_accepts`). UPPAAL rejects lower-bound invariants, and their only role is to make infeasible transitions visible: after forward and backward propagation, a transition whose source location bounds a clock from below and whose target bounds it from above carries both bounds in its guard.

c: Simplification.

Three simplifications follow. First, a transition whose guard contains contradictory constraints is removed: a lower bound and an upper bound on the same clock with $u < l$, or with

$u = l$ and one of them strict, or an upper bound $x \leq u$ with $u < 0$ or $x < u$ with $u \leq 0$. Such a transition is never enabled (Coq: `remove_contradictory_accepts`). Second, the transitions leaving a non-initial location that has no entering transition are removed: such a location never occurs on a run after position 0 (Coq: `remove_unreachable_accepts`). The iteration of the pipeline removes chains of such locations one step at a time. A location whose only entering transitions are self-loops keeps them, but it is unreachable from the initial location; the tool counts and prints only the locations reachable from the initial one and their transitions. Third, a transition whose action label is unsatisfiable, because it requires two different actions or an action and its negation, is removed (Coq: `remove_unsat_accepts`). Such labels arise because the translator treats an action a and the atom $\neg a$ of $\mathcal{T}(f)$ as independent propositions; removing them also enables clock merging, since the resets of a transition that can never be taken would otherwise prevent it.

D. DEAD RESETS AND MERGING OF SYNCHRONOUSLY RESET CLOCKS

a: Dead resets.

A clock x is *live* at a location ℓ if some transition that tests x in its guard can be taken from ℓ , possibly after other transitions. The live locations are computed as a backward closure, along all transitions, from the sources of the transitions that test x ; the other locations are *dead* for x . Since the correctness of the pass does not depend on how this closure is computed, the pass checks the two properties that the proof uses, namely that no transition leads from a dead location to a live one and that none tests x from a dead location, and then removes x from the reset sets of the transitions that enter a dead location. After such a transition, the value of x is never read again, so whether it is reset does not matter. The proof builds, from every accepted extended word, an extended word that differs only in the reset decisions of x , and conversely (Coq: `remove_dead_resets_clock_accepts`). The pass is applied to every clock (`remove_dead_resets_accepts`).

b: Synchronous clocks.

The verified translation shares the clock of syntactically identical timed subformulas. Clocks of *different* subformulas can also be merged when they are reset synchronously. Two clocks $x \neq y$ are *mergeable* if

- (i) every transition resets x if and only if it resets y ;
- (ii) no transition enters the initial location; and
- (iii) every transition leaving the initial location resets x and does not test it.

Then, after the first event, x and y always have the same value, and every test of x can be replaced by the same test on y (Coq: `merge_accepts`). The condition is checked by a Boolean function proved to imply it, and the pass tries all pairs of reset clocks (`merge_all_accepts`). The merged clock x is no longer tested, so its remaining resets are removed by the dead-reset pass in the next round. This verified merging is restricted

to synchronously reset clocks; merging clocks whose values coincide for other reasons is not covered.

E. NORMALIZATION OF GUARDS

Backward and forward propagation add constraints to guards. Every time a guard is strengthened, it is *tightened*: a constraint implied by another constraint of the same guard on the same clock is dropped, so that a guard keeps at most one lower bound and one upper bound per clock (an equality counting as both); for instance, $x \leq 5 \wedge x < 2$ becomes $x < 2$. The implication test compares bounds and strictness, and is proved sound (Coq: `tighten_holds`). Normalization tightens every guard and also removes the lower bounds $x \geq m$ with $m \leq 0$ and $x > m$ with $m < 0$, which every clock value satisfies; it is needed after the merging of clocks, which renames constraints. The removal of trivial lower bounds relies on the nonnegativity of clock values, so the theorem is stated on timed words (Coq: `normalize_accepts`).

F. MERGING OF TRANSITIONS AND OF LOCATIONS

a: Transitions.

Two transitions with the same source, target, and resets are merged when the union of their enabling conditions is again a conjunction. A transition is removed when another transition with the same source, target, and resets is *weaker*, that is, when its label and guard imply those of the other transition, constraint by constraint (subsumption). Two transitions whose labels, or guards, differ only by a literal and its negation, such as p and $\neg p$, $x < 2$ and $x \geq 2$, or $x \leq 5$ and $x > 5$, are replaced by one transition without that literal (resolution). The pass preserves the language because every enabled transition has an enabled counterpart with the same source and target, and conversely (Coq: `merge_trans_accepts`). The other unions are disjunctive; they are formed by the export (Section S1-G).

b: Locations.

A location ℓ' is merged into a location ℓ when both are accepting or both are not, and when every transition leaving ℓ' has a twin leaving ℓ , and conversely: same label, guard, and resets, compared as sets, and same target once ℓ' is replaced by ℓ . The transitions entering ℓ' are redirected to ℓ , and those leaving ℓ' are removed. A run of the original automaton becomes a run of the merged one by replacing ℓ' by ℓ ; conversely, a run of the merged automaton is followed in the original one, taking in ℓ' the twin of the transition taken in ℓ (Coq: `merge_states_all_accepts`; “state” in a Coq name means location). The condition on the Büchi marking is necessary: merging an accepting location with a non-accepting twin would accept runs that visit only the latter infinitely often.

G. TRANSITION MERGING AND ELIMINATION OF DISJUNCTIVE INVARIANTS

The export works on a model whose guards are in disjunctive normal form over clock constraints and clock-difference

constraints $x - y \leq b$, and whose location invariants are disjunctions of conjunctions of upper bounds. It is interpreted on the same trace model as before. The export merges transitions, splits locations, splits disjunctive guards, normalizes the result, and merges the initial location with an equivalent one, each pass being proved to preserve the language of timed words; it ends with an automaton whose guards and invariants are conjunctive, as UPPAAL requires (Coq: `export_accepts`, `export_guards_conjunctive`, and `export_invariants_conjunctive`).

a: Transition merging.

Transitions with the same source, target, action label, and reset set are merged into one transition whose guard is the disjunction of their guards (Coq: `merge_transitions_accepts`). The invariant of a location ℓ is then naturally disjunctive: it has one disjunct $ub(g)$, the conjunction of the upper bounds of g , for every conjunct g of every guard leaving ℓ :

$$I = I_1 \vee \dots \vee I_r.$$

A disjunct that implies another disjunct is redundant and is dropped. The resulting invariant is more precise than the clock-by-clock maximum of Section S1-B, which is its conjunctive over-approximation.

b: Splitting locations.

UPPAAL does not allow disjunctive location invariants: invariants must be conjunctions of clock constraints defining convex, past-closed zones. A location with invariant $I_1 \vee \dots \vee I_r$ is therefore replaced by locations with conjunctive invariants and the same timed behavior. Bodeveix and Filali studied disjunctive invariants in UPPAAL [2]. Figure 2 in the article summarizes the transformation.

The location L is replaced by one copy L_k for each disjunct I_k , with the conjunctive invariant I_k . Every transition entering L is replicated toward every copy L_k , and its guard is strengthened with the entry condition Ent_k after the resets; every transition leaving L is replicated from every copy. The disjunctive invariant itself is never built: the copies introduce the conjunctive invariants directly, and the theorem compares the split automaton with the automaton without invariants. The run starts in a dedicated copy of the initial location, without an invariant, since it is not entered by a transition. In the Coq development, the copy k of location ℓ is numbered $\ell \cdot \kappa + k$, where $\kappa - 1$ is the largest number of disjuncts and the copy $\kappa - 1$ is the initial one; numbers that correspond to no disjunct are unreachable locations.

c: Entry condition.

The disjuncts need not be mutually exclusive, so simple replication would introduce nondeterministic choices and could select a copy that permits less time to elapse than another one. The entry condition therefore selects a copy with the *longest*

admissible delay. The synthesized invariants have non-strict bounds only (Section S1-B); let

$$I_k = \bigwedge_{c \in C_k} c \leq m_c^k.$$

From a valuation v , I_k admits the delays up to $Del_k(v) = \min_{c \in C_k} (m_c^k - v(c))$, with $Del_k(v) = +\infty$ when C_k is empty. The entry condition of L_k requires I_k and $Del_l(v) \leq Del_k(v)$ for every other copy l . The comparison $Del_l \leq Del_k$ holds when some bound $c' \in C_l$ is reached no later than every bound $c \in C_k$, which is expressed with clock differences:

$$Ent_k \triangleq I_k \wedge \bigwedge_{l \neq k} \bigvee_{c' \in C_l} \bigwedge_{c \in C_k} (c - c' \leq m_c^k - m_{c'}^l).$$

When C_k is empty, the copy admits every delay and, since redundant disjuncts are removed beforehand, no other C_l is empty, so that the conjunction over l is true. Ties are allowed: when two copies admit the same delay, both entry conditions hold.

Theorem S1.3 (Elimination of disjunctive invariants). *The split automaton accepts the same timed words as the automaton without invariants.*

Proof. Every copy L_k behaves as L with additional constraints, so the split automaton accepts no new word. Conversely, consider an accepting run that enters L with the valuation v (after the resets) and leaves it after a delay τ along a transition whose guard has a conjunct g . This conjunct holds at $v + \tau$, hence so does the disjunct $ub(g) = I_j$, and $Del_j(v) \geq \tau$. If I_j was dropped because it implies a kept disjunct, that disjunct admits at least the same delays, so we may assume that I_j is kept (Coq: `disj_inv_cover`). Choose a copy k whose delay $Del_k(v)$ is maximal (Coq: `best`). Its entry condition holds at v , and $Del_k(v) \geq Del_j(v) \geq \tau$, so I_k holds during the whole stay. The successor copies are chosen in the same way at every position. Clock-difference constraints are invariant under the elapse of time, so Ent_k can be checked on the valuation after the resets, and $Ent_k[Z := 0]$ on the valuation before them. (Coq: `split_accepts`.) $\square$

The substitution $[Z := 0]$ maps a difference $c - c' \leq b$ to $0 \leq b$ when both clocks are reset, to $c' \geq -b$ when only c is reset, and to $c \leq b$ when only c' is reset. The disjunctions over c' in Ent_k , and those created by transition merging, are finally eliminated by putting the guards into disjunctive normal form and splitting every transition into one copy per disjunct (Coq: `explode_accepts`). In the worst case, the number of copies is the product of the sizes of the disjunctions.

d: Final normalization.

The guards are normalized before and after the split (Coq: `normalize_d_accepts`): in every conjunction, constraints implied by another one are removed, as in Section S1-E, together with the trivial differences $x - x \leq b$ with $b \geq 0$; conjunctions containing a difference $x - x \leq b$

with $b < 0$ are unsatisfiable and are removed; and a conjunction that implies another conjunction of the same guard is removed. Finally, the transitions leaving a location that is neither initial nor the target of a transition are removed, until nothing changes (Coq: `dprune_n_accepts`); this removes in particular the transitions of the copies that correspond to no disjunct. The dedicated copy of the initial location is then merged into another location when both have the same acceptance status, equivalent invariants (an invariant with an empty disjunct being equivalent to no invariant), and the same outgoing transitions, compared as sets after redirection; this is the case, for instance, when the initial location has no invariant (Coq: `dmerge_states_all_accepts`). The resulting automaton has conjunctive guards with difference constraints and conjunctive location invariants made of upper bounds.

e: Symbolic transitions.

Each transition of the exported automaton reads a cube: a conjunction of action literals and a conjunction of clock constraints. For the drawing and for the comparison with tools whose transitions carry Boolean formulas, the last pass merges all the transitions with the same source, target, and resets into one *symbolic transition*, labeled by a disjunction of cases $E_1 \wedge c_1 \vee \dots \vee E_k \wedge c_k$, where E_j is a set of events, either finite or the complement of a finite set, and c_j a conjunction of clock constraints; the cases with the same constraints are merged by union of their sets of events. A symbolic transition is enabled when the current event belongs to some E_j whose constraints c_j hold, and its resets match those of the extended word. The symbolic automaton accepts the same extended words as the exported one, hence the same timed words for both the existential and the conventional acceptance (Coq: `symbolic_ext_accepts`, `MTL_to_symbolic_correct_with`, and `MTL_to_symbolic_correct0_with`). The UPPAAL model keeps the conjunctive transitions, since a UPPAAL edge synchronizes on one channel and its clock guard must be a conjunction.

52. EVALUATION ARTIFACTS AND EXTENDED MEASUREMENTS

This section records benchmark rows and input formulas used in the article. Source tables and raw data are included under `evaluation/`; the values below are tracked artifacts.

A. BENCHMARK FORMULAS

Table S1 lists the 31 configurations. Rn is the conjunction of n independent bounded responses, and Nn is the nested bounded-eventuality family. These formula-generated inputs support tool evaluation; they do not define the domain of the generic Coq theorems.

B. COMPLETE STRUCTURAL EVALUATION

The full table reports the upstream Spot automaton, optimized automaton, UPPAAL export, symbolic representation, wall-clock times, CASAAL sizes, and language-comparison out-

come. For comparisons, `exp` means an inclusion was established on the exported automaton; `cmp` means an inclusion was established only on the compiled automaton before post-processing; `?` means no conclusion within limits; `-` means the formulas are not equivalent because hatted operators have no CASAAL counterpart. Equal sizes do not establish language equality.

C. SCALING AND WEAK-UNTIL ABLATION

The scaling table reports stage times, min/max over five runs, peak memory, and exported transition counts for the scalable families. The second table compares the tracked weak-until default with the measurement-only `-noweak` option. These results describe Spot 2.16 and the current configuration; they are not asymptotic bounds.

D. TRANSLATION VALIDATION AND LANGUAGE COMPARISONS

The evaluation repository includes scripts and raw TSV results for checks summarized in Section VI:

- **Spot-output cross-check.** An independent translator (ltl2ba 1.3) and Spot's `autfilt` check the returned automaton against the clocked-LTL input. Both inclusions conclude on 25 configurations; on R6–R8 and N6–N8, ltl2ba does not finish within 300 seconds. This is benchmark translation validation, not a verified replacement for the LTL-to-Büchi translator.
- **Interface check.** With `-check`, Spot parses the formula from the ordinary printer and a second prefix printer, and `autfilt -equivalent-to` compares the automaton read by the tool with Spot's raw output. Checks pass on all 31 configurations. A common error in both printers or Spot's parser is not ruled out.
- **CASAAL products.** TChecker 0.8 checks products of the two tool automata for a formula and its negation, with event exclusivity, increasing timestamps, and divergent time enforced by additional processes. Its conversion script is unverified. The 29 comparable configurations yield 17 conclusions on exported automata, 3 on the pre-optimization compiled automaton only, and 9 inconclusive cases.
- **Random formulas.** The repository records 100 generated formulas of depth at most three over three propositions. With a 120-second bound per product check, comparison products are empty for 95 formulas; for four others, checks that finish report emptiness. One remaining formula produces a nonempty product related to CASAAL's printed acceptance marking for an unfulfilled eventuality, as documented in `evaluation/README.md`.
- **Implementation checks.** Eighteen finite-word simulator checks exercise strict and non-strict boundaries and overlapping activations. They pass and validate selected output behaviors; they do not replace the mechanized proof.

53. IMPLEMENTATION AND REPRODUCTION DETAILS

The verified functions are defined in the local `proof/tree` and extracted into `tool/src/optim.ml`. The project

TABLE S1. Benchmark formulas, in the input syntax of `mtl2tba` ($\neg$, $\wedge$, $\vee$, $\rightarrow$, $\rightarrow$: hatted operators; $[]$, $\langle \rangle$, $\square$, $\diamond$; $U[\leq 2]$: Until with bound ≤ 2 ; $!$, $\&$, $|$, $\rightarrow$: negation, conjunction, disjunction, implication). F3 and F3b use the non-strict bound ≤ 2 . CASAAL receives the same formula, with $\wedge$ and $\vee$ for $\wedge$ and $\vee$, conjoined with $[]$ ($!(a \wedge b)$) for every pair of distinct propositions; for F3 and F13 it receives the closest encodings with a leading $\diamond$, which are not equivalent. R1 is F1 with renamed propositions; it is the first instance of the family Rn .

Id	Property	Formula
F1	bounded response	$[] (p \rightarrow \langle \rangle [\leq 5] q)$
F2	bounded invariance	$[] (p \rightarrow [] [\leq 3] q)$
F3	sporadicity (hatted)	$[] (e \rightarrow \hat{[]} [\leq 2] !e)$
F3b	sporadicity, no hat	$[] (e \rightarrow [] [\leq 2] !e)$
F4	time-bounded Until	$[] (p \rightarrow (p U[\leq 4] q))$
F5	minimum-delay response	$[] (p \rightarrow \langle \rangle [\geq 2] q)$
F6	strict minimum-delay response	$[] (p \rightarrow \langle \rangle [\geq 2] q)$
F7	bounded recurrence	$[] \langle \rangle [\leq 5] p$
F8	recurrence with minimum delay	$[] \langle \rangle [\geq 2] p$
F9	Example S1	$\langle \rangle [\leq 5] (p U[\leq 2] q)$
F10	Example S2	$\langle \rangle [\leq 5] (([] [\leq 3] (p q)) U[\leq 2] q)$
F11	Running example	$(\langle \rangle [\leq 5] p) \& ([] [\leq 2] !p)$
F12	response within a window	$[] (p \rightarrow (\langle \rangle [\leq 5] q \& [] [\leq 2] !q))$
F13	periodicity (hatted)	$e \& [] (e \rightarrow (\hat{[]} [\leq 2] !e \& \langle \rangle [\leq 2] e))$
F14	closeness	$[] (e1 \rightarrow (\langle \rangle [\leq 2] (e2 \& \langle \rangle [\leq 3] e3) [] (!e2 \& !e3)))$
R1	1 bounded response	$[] (a1 \rightarrow \langle \rangle [\leq 5] b1)$
R2	2 bounded responses	$[] (a1 \rightarrow \langle \rangle [\leq 5] b1) \& [] (a2 \rightarrow \langle \rangle [\leq 5] b2)$
R3	3 bounded responses	$[] (a1 \rightarrow \langle \rangle [\leq 5] b1) \& [] (a2 \rightarrow \langle \rangle [\leq 5] b2) \& [] (a3 \rightarrow \langle \rangle [\leq 5] b3)$
R4	4 bounded responses	$[] (a1 \rightarrow \langle \rangle [\leq 5] b1) \& [] (a2 \rightarrow \langle \rangle [\leq 5] b2) \& [] (a3 \rightarrow \langle \rangle [\leq 5] b3) \& [] (a4 \rightarrow \langle \rangle [\leq 5] b4)$
R5	5 bounded responses	$[] (a1 \rightarrow \langle \rangle [\leq 5] b1) \& [] (a2 \rightarrow \langle \rangle [\leq 5] b2) \& [] (a3 \rightarrow \langle \rangle [\leq 5] b3) \& [] (a4 \rightarrow \langle \rangle [\leq 5] b4) \& [] (a5 \rightarrow \langle \rangle [\leq 5] b5)$
R6	6 bounded responses	$[] (a1 \rightarrow \langle \rangle [\leq 5] b1) \& [] (a2 \rightarrow \langle \rangle [\leq 5] b2) \& [] (a3 \rightarrow \langle \rangle [\leq 5] b3) \& [] (a4 \rightarrow \langle \rangle [\leq 5] b4) \& [] (a5 \rightarrow \langle \rangle [\leq 5] b5) \& [] (a6 \rightarrow \langle \rangle [\leq 5] b6)$
R7	7 bounded responses	$[] (a1 \rightarrow \langle \rangle [\leq 5] b1) \& [] (a2 \rightarrow \langle \rangle [\leq 5] b2) \& [] (a3 \rightarrow \langle \rangle [\leq 5] b3) \& [] (a4 \rightarrow \langle \rangle [\leq 5] b4) \& [] (a5 \rightarrow \langle \rangle [\leq 5] b5) \& [] (a6 \rightarrow \langle \rangle [\leq 5] b6) \& [] (a7 \rightarrow \langle \rangle [\leq 5] b7)$
R8	8 bounded responses	$[] (a1 \rightarrow \langle \rangle [\leq 5] b1) \& [] (a2 \rightarrow \langle \rangle [\leq 5] b2) \& [] (a3 \rightarrow \langle \rangle [\leq 5] b3) \& [] (a4 \rightarrow \langle \rangle [\leq 5] b4) \& [] (a5 \rightarrow \langle \rangle [\leq 5] b5) \& [] (a6 \rightarrow \langle \rangle [\leq 5] b6) \& [] (a7 \rightarrow \langle \rangle [\leq 5] b7) \& [] (a8 \rightarrow \langle \rangle [\leq 5] b8)$
N1	nesting depth 1	$\langle \rangle [\leq 5] p1$
N2	nesting depth 2	$\langle \rangle [\leq 5] (p1 \& \langle \rangle [\leq 5] p2)$
N3	nesting depth 3	$\langle \rangle [\leq 5] (p1 \& \langle \rangle [\leq 5] (p2 \& \langle \rangle [\leq 5] p3))$
N4	nesting depth 4	$\langle \rangle [\leq 5] (p1 \& \langle \rangle [\leq 5] (p2 \& \langle \rangle [\leq 5] (p3 \& \langle \rangle [\leq 5] p4)))$
N5	nesting depth 5	$\langle \rangle [\leq 5] (p1 \& \langle \rangle [\leq 5] (p2 \& \langle \rangle [\leq 5] (p3 \& \langle \rangle [\leq 5] (p4 \& \langle \rangle [\leq 5] p5))))$
N6	nesting depth 6	$\langle \rangle [\leq 5] (p1 \& \langle \rangle [\leq 5] (p2 \& \langle \rangle [\leq 5] (p3 \& \langle \rangle [\leq 5] (p4 \& \langle \rangle [\leq 5] (p5 \& \langle \rangle [\leq 5] p6))))))$
N7	nesting depth 7	$\langle \rangle [\leq 5] (p1 \& \langle \rangle [\leq 5] (p2 \& \langle \rangle [\leq 5] (p3 \& \langle \rangle [\leq 5] (p4 \& \langle \rangle [\leq 5] (p5 \& \langle \rangle [\leq 5] (p6 \& \langle \rangle [\leq 5] p7))))))$
N8	nesting depth 8	$\langle \rangle [\leq 5] (p1 \& \langle \rangle [\leq 5] (p2 \& \langle \rangle [\leq 5] (p3 \& \langle \rangle [\leq 5] (p4 \& \langle \rangle [\leq 5] (p5 \& \langle \rangle [\leq 5] (p6 \& \langle \rangle [\leq 5] (p7 \& \langle \rangle [\leq 5] p8))))))$

Makefile compiles the Rocq development and extracts the executable functions. Hand-written components include the parser, Spot invocation, LBTT reader, iteration driver, and XML/DOT printers. The script that removes unreachable extracted definitions and the Rocq/OCaml toolchains are part of the implementation trusted base.

The data correspond to five runs per configuration, with a 300-second per-run limit. The machine record is in `evaluation/machine.txt`; raw run files, median summaries, table generator, benchmark formulas, case-study models, and validation scripts are included. Compile instruc-

tions are in `README.md`.

The Coq transformations are verified under existential initial clock values. The zero-initialization check used by the formula-generated workflow is not a generic input-format guarantee. Using an arbitrary external TBA requires a separate argument that its initialization and printed representation match the intended UPPAAL semantics.

S4. UPPAAL CASE-STUDY MODELS

This section records the four small system models behind the demonstration in Section VI. Model files, queries, and

TABLE S2. Evaluation of `mt12tba` and comparison with CASAAL on the common semantic domain (one event per position). $|X|$: number of distinct primitive (hatted) timed subformulas after unfolding of the ordinary operators and the default rewritings. Spot: Büchi automaton returned by Spot (states/transitions, one transition per cube). Opt.: after reset completion and the verified optimizations (locations/transitions/clocks). Export: automaton for UPPAAL (locations, transitions, clocks, locations with an invariant, and occurrences of difference constraints in the guards). Sym.: transitions of the symbolic automaton, with Boolean labels. Time: wall-clock time, median of five runs of the whole chain including Spot, in seconds. st, tr, loc, clk: states, transitions, locations, clocks. CASAAL: states/transitions/clocks of the automaton of CASAAL for the formula conjoined with the mutual exclusion of its propositions; transitions carry Boolean formulas. †: not equivalent (CASAAL has no hatted operator); ‡: degenerate formula, equivalent to $\Box \neg p$ or $\Box \neg e$ with one event per position. CASAAL time: wall-clock time, median of five runs, in seconds, including the start of the process. TO: the chain exceeds the limit of 300 s (wall-clock, per run). Lang.: comparison of the languages with those of CASAAL by TChecker (Section VI, paragraph “Comparison of languages with CASAAL”): exp, inclusion of the languages of CASAAL’s automata into ours established on the exported automaton; cmp, established only on the automaton `compile(f)` before post-processing; ?, no conclusion within the limits; –, formulas not equivalent. Equal sizes do not by themselves imply equal languages. Formulas: Table S1 of the supplementary material.

Id	$ X $	Spot		Opt.			Export					Sym.	Time	CASAAL		Lang.
		st	tr	loc	tr	clk	loc	tr	clk	inv	diff			st/tr/clk	Time	
F1	1	2	5	2	5	1	2	5	1	1	0	4	0.02	2/4/1	0.05	exp
F2 [†]	1	2	5	1	1	0	1	1	0	0	0	1	0.03	1/1/0	0.06	exp
F3	1	2	5	2	5	1	2	5	1	0	0	5	0.02	3/6/1 [†]	0.05	–
F3b [†]	1	1	1	1	1	0	1	1	0	0	0	1	0.02	1/1/0	0.05	exp
F4	1	2	5	2	5	1	2	5	1	1	0	4	0.02	2/4/1	0.06	exp
F5	1	7	38	7	25	1	7	25	1	0	0	25	0.04	4/15/1	0.05	exp
F6	1	7	38	7	25	1	7	25	1	0	0	25	0.08	4/15/1	0.05	exp
F7	1	2	4	2	4	1	2	4	1	1	0	4	0.03	2/4/1	0.05	exp
F8	0	2	4	2	4	0	2	4	0	0	0	4	0.02	2/4/1	0.06	exp
F9	2	4	9	4	9	2	4	9	2	2	0	9	0.03	4/9/2	0.06	exp
F10	3	5	13	5	13	3	5	13	3	2	0	12	0.04	5/12/3	0.05	exp
F11	2	5	11	4	7	1	4	7	1	2	0	7	0.03	4/7/2	0.05	exp
F12	2	4	17	3	9	2	3	9	2	2	0	9	0.02	3/9/2	0.05	exp
F13	2	5	17	3	8	2	3	8	2	2	0	8	0.02	4/7/2 [†]	0.05	–
F14	2	6	26	6	17	2	6	19	2	4	2	14	0.03	8/18/2	0.06	cmp
R1	1	2	5	2	5	1	2	5	1	1	0	4	0.02	2/4/1	0.05	exp
R2	2	4	25	4	16	2	4	19	2	3	2	12	0.03	4/12/2	0.06	cmp
R3	3	8	125	8	44	3	8	67	3	7	30	32	0.07	8/32/3	0.06	cmp
R4	4	16	625	16	112	4	16	217	4	15	192	80	0.41	16/80/4	0.10	?
R5	5	32	3125	32	272	5	32	651	5	31	880	192	2.90	32/192/5	0.19	?
R6	6	64	15625	64	640	6	64	1837	6	63	3360	448	36.75	64/448/6	0.58	?
R7	7							TO						128/1024/7	1.73	?
R8	8							TO						256/2304/8	4.86	?
N1	1	3	5	3	5	1	3	5	1	1	0	5	0.02	3/5/1	0.05	exp
N2	2	4	11	4	9	2	4	9	2	2	0	7	0.02	4/7/2	0.08	exp
N3	3	5	21	5	16	3	5	16	3	3	0	9	0.03	5/9/3	0.05	exp
N4	4	6	36	6	25	4	6	25	4	4	0	11	0.06	6/11/4	0.05	exp
N5	5	7	57	7	36	5	7	36	5	5	0	13	0.36	7/13/5	0.06	?
N6	6	8	85	8	49	6	8	49	6	6	0	15	1.21	8/15/6	0.06	?
N7	7	9	121	9	64	7	9	64	7	7	0	17	10.94	9/17/7	0.07	?
N8	8	10	166	10	81	8	10	81	8	8	0	19	103.02	10/19/8	0.07	?

recorded outputs are in `evaluation/case_study/`. Edges below use the form: source $\rightarrow$ target: guard, event / resets; an omitted guard is true.

A. PROPERTY AUTOMATA

For sporadicity, the violation formula $\Diamond(e \wedge \widehat{\Diamond}_{<2} e)$ is compiled into locations $L0$ (initial), $A2$ (accepting, invariant $x_0 \leq 2$), and accepting sink $A4$, with one clock:

- $L0 \rightarrow L0$: every event / x_0 ; $L0 \rightarrow A2$: e / x_0 ;
- $A2 \rightarrow A2$: $x_0 < 2$, other; $A2 \rightarrow A4$: $x_0 < 2$, e ;
- $A4 \rightarrow A4$: every event.

The automaton guesses one e and reaches $A4$ when a second follows less than two time units later. The bounded accepting location cannot be occupied forever on a time-divergent word, so acceptance entails a visit to the violation sink.

For bounded response, the violation formula is $\Diamond(p \wedge \widehat{\Box}_{\leq 5} \neg q)$, with locations $L0$ (initial), $A2$ (accepting), and $A4$ (accepting sink):

- $L0 \rightarrow L0$: every event; $L0 \rightarrow A2$: p / x_0 ;
- $A2 \rightarrow A2$: p or other; $A2 \rightarrow A4$: $x_0 > 5$, every event;
- $A4 \rightarrow A4$: every event.

A run that remains in $A2$ forever corresponds to a request that is never answered. On a word with divergent time and infinitely many events, an event eventually enables the transition to $A4$.

B. SYSTEM MODELS AND RESULTS

Every system has a clock z , reset on every emission and tested by $z > 0$ so event times strictly increase, and a clock y for its timing parameter.

TABLE S3. Scalability on the families Rn and Nn : median time of the Spot, optimization, and export stages and of the whole chain, minimum and maximum of the five runs, peak memory, and exported transitions with Spot options $-B$ $-small$ (default) and $-B$ $-D$. Times in seconds, memory in MB. The total also includes the stages not listed: reading of the output of Spot, reset completion, check $init_free$, symbolic transitions, and printing.

Id	Spot	Opt.	Exp.	Total	[min, max]	Mem.	tr	tr -D
R1	0.01	0.00	0.00	0.02	[0.02, 0.05]	17	5	5
R2	0.01	0.00	0.00	0.03	[0.02, 0.06]	17	19	19
R3	0.03	0.01	0.02	0.07	[0.06, 0.20]	18	67	67
R4	0.07	0.04	0.18	0.41	[0.30, 0.49]	19	217	217
R5	0.48	0.68	1.12	2.90	[2.61, 3.44]	23	651	651
R6	6.36	16.57	10.31	36.75	[36.53, 39.80]	106	1837	1837
R7				timeout (limit 300 s)				
R8				timeout (limit 300 s)				
N1	0.01	0.00	0.00	0.02	[0.02, 0.04]	17	5	5
N2	0.01	0.00	0.00	0.02	[0.02, 0.04]	17	9	26
N3	0.01	0.00	0.00	0.03	[0.03, 0.05]	18	16	173
N4	0.04	0.00	0.00	0.06	[0.05, 0.07]	19	25	1035
N5	0.31	0.00	0.02	0.36	[0.32, 0.61]	22	36	5461
N6	1.08	0.02	0.05	1.21	[1.18, 1.25]	35	49	timeout
N7	10.47	0.07	0.21	10.94	[9.70, 13.26]	140	64	timeout
N8	101.42	0.24	0.64	103.02	[101.06, 104.84]	961	81	timeout

TABLE S4. Effect of the weak until (default; disabled by the option $-noweak$), on the configurations whose sizes change. Spot: states of the automaton returned by Spot; Sym.: symbolic transitions; Time: median of five runs, in seconds; TO: more than 300 s; CASAAL: states/transitions on the common domain, [†] as in Table S2. The clocks do not change.

Id	$-noweak$			default			CASAAL
	Spot	Sym.	Time	Spot	Sym.	Time	
F1	3	6	0.02	2	4	0.02	2/4
F4	3	6	0.03	2	4	0.02	2/4
F7	3	6	0.02	2	4	0.03	2/4
F12	5	13	0.05	4	9	0.02	3/9
F13	6	9	0.03	5	8	0.02	4/7 [†]
F14	8	20	0.04	6	14	0.03	8/18
R1	3	6	0.02	2	4	0.02	2/4
R2	8	21	0.04	4	12	0.03	4/12
R3	20	68	0.23	8	32	0.07	8/32
R4	48	205	2.14	16	80	0.41	16/80
R5	112	582	37.48	32	192	2.90	32/192
R6		TO		64	448	36.75	64/448

- *Sporadic emitter.* A single location S has $S \rightarrow S: y \geq k \wedge z > 0, e / y, z$, and $S \rightarrow S: z > 0, other / z$. It satisfies the requirement for $k = 2$ and violates it for $k = 1$.
- *Controller.* Locations are *Idle* (initial) and *Busy*, with invariant $y \leq d$. The transition $Idle \rightarrow Busy$ emits p when $z > 0$ and resets y, z ; *Idle* can emit *other* when $z > 0$ and reset z . In *Busy*, *other* is emitted under $z > 0 \wedge y < d$, resetting z , and q is emitted under $z > 0 \wedge y \geq 1$, resetting z and returning to *Idle*. It satisfies bounded response for $d = 4$ and violates it for $d = 6$.

The query is $E \langle \rangle P.A4$. Table S5 records UPPAAL 5.0.0 results. The system-alone deadlock check composes each system with a receiver of every channel, without the property automaton. A controller variant without guard $y < d$ can emit *other* at $y = d$, after which the $z > 0$ test prevents another event while the invariant prevents time advancing.

TABLE S5. Case study with UPPAAL 5.0.0. Reachability is $E \langle \rangle P.A4$; no deadlock is $A[] \text{ not deadlock}$ on the system alone.

System	Reachability	Diagnostic trace	No deadlock
Emitter, $k = 2$	no	—	yes
Emitter, $k = 1$	yes	$(e, 1)(e, 2)$	yes
Controller, $d = 4$	no	—	yes
Controller, $d = 6$	yes	$(p, 0.5)(q, 6)$	yes

UPPAAL's system-alone $A[] \text{ not deadlock}$ query detects this timelock; packaged models use the corrected guard.

REFERENCES

- [1] G. Behrmann, A. David, and K. G. Larsen, "A tutorial on UPPAAL," in *Formal Methods for the Design of Real-Time Systems*, ser. Lecture Notes in Computer Science. Springer, 2004, vol. 3185, pp. 200–236.
- [2] J.-P. Bodeveix and M. Filali, "About disjunctive invariants in UPPAAL (NWPT, Turku Finlande, 29/10/03–31/10/03)," in *NWPT, Turku Finlande, 29/10/03–31/10/03*, octobre 2003. [Online]. Available: <http://www.abo.fi/~Kaisa.Sere/15NWPT.html>

...